Unit - 3

Inheritance, Interface and Package
(12 Marks)

INHERITANCE

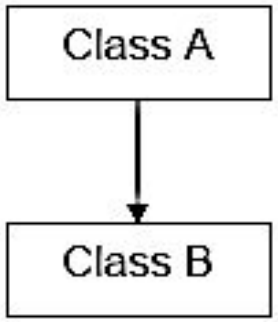
- **The mechanism of deriving a new class from existing one (base class) is called as inheritance.**
- **Derived class can inherit the properties(variables & methods) from base class while adding new feature in its own class**
- **Reusability is main feature in java**
- **New class derived from base class with keyword 'extends'**
- **Child class can access members and methods of its parent class**
- **Avoid creating multiple object of base class.**
- **We only create object of Derived class which serve function of derived as well as parent class**

TYPES-INHERITANCE

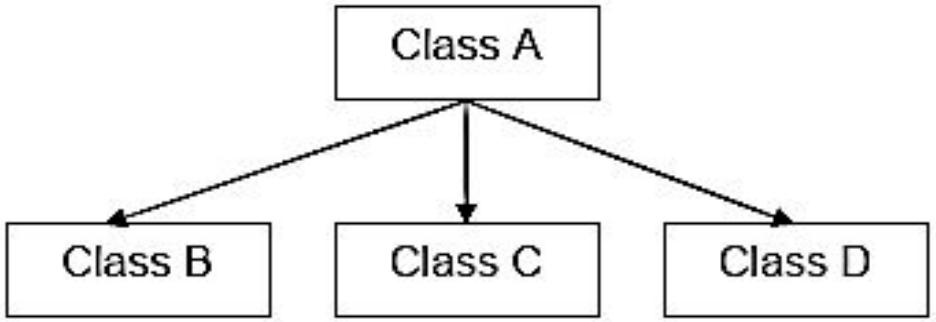
Single	One super and one sub class
Multi level	Intermediate class between super and sub class (More than one level)
Multiple	More than 1 super and 1 sub class
Hierarchical	1 super and more than one sub class
Hybrid	Combination of any

NOTE: **Multiple Inheritance not applicable** in java, to achieved this java provide the concept of ***Interface***

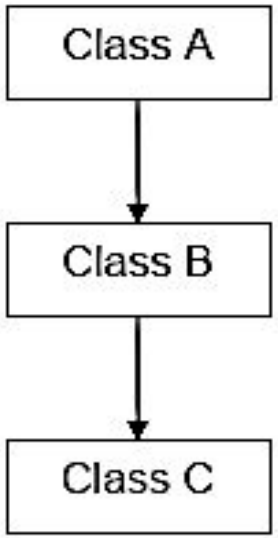
TYPES-INHERITANCE



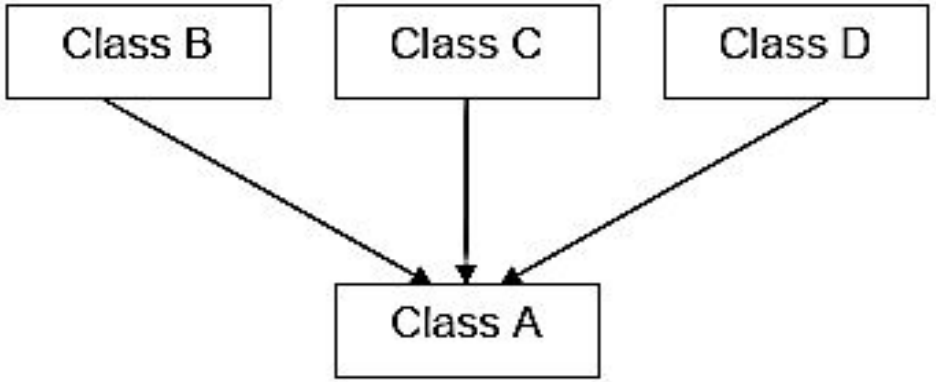
a). Single Inheritance



b) Hierarchical Inheritance

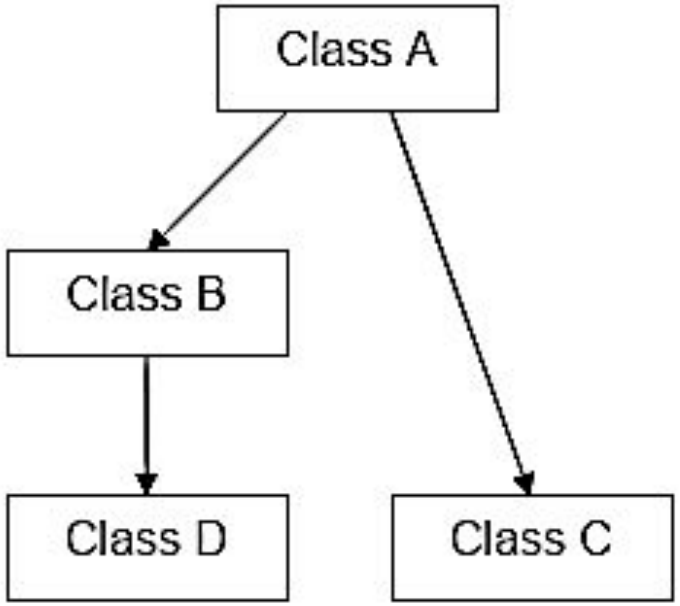


c) Multilevel Inheritance



d) Multiple Inheritance

TYPES-INHERITANCE



e) Hybrid Inheritance

Fig. 2.2 Types of Inheritance

SYNTAX-INHERITANCE

Inherit properties of super class into sub class

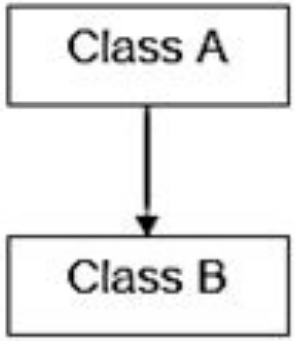
Syntax…

```
class Parent
{
    ---
}
class Derived extends Parent
{
    ---
}
```

keyword

SYNTAX- SINGLE LEVEL INHERITANCE

```
class Parent
{
        ----
        ----
}
class Child extends Parent
{
        ----
        ----
}
class Family
{
        Public static void main(String args[])
        {
                Child C=new Child();
        }
}
```



EXAMPLE- SINGLE LEVEL INHERITANCE

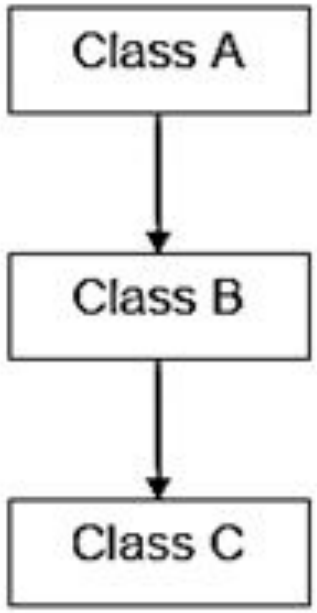
```
** Single Level Inheritance **

class Parent
{                    // Super class
  int i=10;
  void display()
  {
     System.out.println("i: "+i);
  }
}
class Child extends Parent
{
  int j;             // Sub class
  void show()
  {
    System.out.println("i+j: "+(i+j));
  }
}

class Inher          // main class
{
  public static void main(String args[])
  {
     Child c=new Child();  // create child object
     c.j=30;
     c.display();          // calling Parents method
     c.show();             // calling Childs method
  }
}
```

```
// An example of Single inheritance
import java.util.Scanner;
class First
{
    int val;
    void init()
    {
        System.out.print("Enter a number: ");
        Scanner in = new Scanner(System.in);
        val = in.nextInt();
    }
    int square()
    {
        return(val*val);
    }
}
class Second extends First
{
    int mem;
    int cube()
    {
        mem = square() * val;
        return mem;
    }
}
class SingleInheritance
{

    public static void main(String args[])
    {
        Second s = new Second();
        s.init();
        System.out.println("Cube : "+s.cube());
    }
}
```

Program 2.18 Example of single inheritance

SYNTAX- MULTI LEVEL INHERITANCE

```
class GrandParent
{
        ----
        ----
}
class Parent extends GrandParent
{
        ----
        ----
}
class Child extends Parent
{
        ----
        ----
}
class Family
{
        Public static void main(String args[])
        {
                Child C=new Child();
        }
}
```



SYNTAX- MULTI LEVEL INHERITANCE

```
class one
{
    int x=10;
    void on()
    {
        System.out.println("x= "+x);
    }
}
class two extends one
{
    int y=20;
    void tw()
    {
        System.out.println("x+y= "+(x+y));
    }
}
class three extends two
{
    int z=30;
    void th()
    {
        System.out.println("x+y+z= "+(x+y+z
    }
}
```

```
class Number1
{
    public static void main(String args[])
    {
        three t=new three();
        t.on();
        t.tw();
        t.th();
    }
}
```

USE OF SUPER KEYWORD

- To refer the variable declared in super class, hidden by sub class the super keyword is used.

  E.g. : super.variablename;

- To call the constructor of immediate super class of sub class super keyword is used.

Rule

1. super () keyword must be the first statement in sub class constructor.
2. Number of parameters and data type of each parameter in super () must be same as super class constructor.

- 3. super() keyword is used to access the method of super

USING SUPER TO CALL SUPERCLASS CONSTRUCTORS

- A subclass can call a constructor defined by its superclass by use of the following form of super:
  - super(arg-list);
- Here, arg-list specifies any arguments needed by the constructor in the superclass. super( ) must always be the first statement executed inside a subclass' constructor.
- super() keyword is used to access the method of super class with same name and parameter declared in both sub and super class.
  - Example super.methodname();

```
class Box
{
double width;
double height;
double depth;
// constructor used when all dimensions specified

Box(double w, double h, double d)
{
width = w;
height = h;
depth = d;
}
}
```

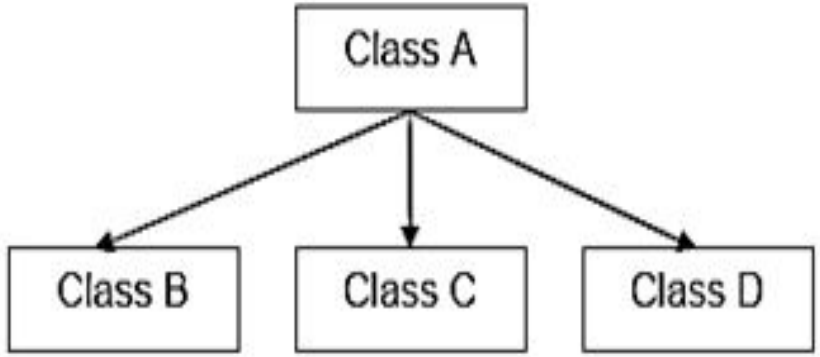
```
class BoxWeight extends Box
{
double weight; // weight of box

// initialize width, height, and depth using super()
BoxWeight(double w, double h, double d, double m)
{
super(w, h, d); // call superclass constructor
weight = m;
}
}
```

SUPER: CALLING CONSTRUCTOR OF SUPER CLASS INTO SUB CLASS

```
class a                                 // parent class
{
        int i;
        a(int y)
        {
                System.out.println ("y= "+y);

        }
}

class b extends a                               // child class
{

        b (int x)
        {
                super(x);

                System.out.println ("x= "+x);
        }
}

class c                                 // main class
{
        public static void main(String ar[])
        {
                b b2=new b(11);
        }
}

OUT PUT: y= 11
         x= 11
```

SYNTAX- HIERARCHICAL INHERITANCE

```
class Parent
{
        ----
        ----
}
class Son extends Parent
{
        ----
        ----
}
class Daughter extends Parent
{
        ----
        ----
}
class Family
{
        Public static void main(String args[])
        {
                Son S=new Son();
                Daughter D=new Daughter
        }
}
```



```
class Person
{
        String name="Mr. Rathod";
        void fun1()
        {
                System.out.println("Person name= "+name);
        }
}
class blood extends Person
{
        String BG="O+ve";
        void fun2()
        {
                //fun1();
                System.out.println("Blood Group= "+BG);
        }
}
class city extends Person
{
        String village="Nashik";
        void fun3()
        {
                //fun1();
                System.out.println("City Name= "+village);
        }
}
class Info
{
        public static void main(String arg[])
        {
                blood b=new blood();
                city c=new city();
                b.fun1();
                b.fun2();
                c.fun1();
                c.fun3();
        }
}
```

VISIBILITY CONTROL / ACCESS SPECIFIER IN INHERITANCE

Access Specifier	Package A		Package B	
	Super class	Sub class	Super class	Sub class
Public	YES	YES	YES	YES
Private	YES	NO	NO	NO
Protected	YES	YES	NO	YES
Friend/ Default	YES	YES	NO	NO

METHOD OVERRIDING

- when a method in a subclass has the same name, same arguments and same return type as a method in its super class, then the method in the subclass is said to ***over-ride*** the method in the super class.

- Method Over-riding is the concept of Same method name in Super as well as Sub class.

- Methods of super class can be called to its sub class using 'super' keyword.

METHOD OVERRIDING

```
class Parent
{
 void show()
  {
     System.out.println("Information Technoology");
  }

}
class Child extends Parent
{
 void show()
  {
     super.show();
     System.out.println("Computer Technology");
  }
}

class Family
{
   public static void main (String ar[])
   {
      Child b1=new Child();
      b1.show();
   }
}
```

Output:

```
        Information Technology
        Computer Technology
```

SUPER: CALLING CONSTRUCTOR OF SUPER CLASS INTO SUB CLASS

```
class a                                 // parent class
{
        int i;
        a(int y)
        {
                System.out.println ("y= "+y);

        }
}

class b extends a                               // child class
{

        b (int x)
        {
                super(x);

                System.out.println ("x= "+x);
        }
}

class c                                 // main class
{
        public static void main(String ar[])
        {
                b b2=new b(11);
        }
}

OUT PUT: y= 11
         x= 11
```

DYNAMIC METHOD DISPATCH

- Dynamic method dispatch resolves a call to an overridden method at run time, rather than compile time.

- It is a run time polymorphism

- A super class reference variable can refer to a subclass object.

- User having rights to called any overridden method at any sequence of call.

```java
//Dynamic method dispatch
class Principal
{
    void message()
    {
        System.out.println("This is Principal");
    }
}
class Professor extends Principal
{
    void message()
    {
        System.out.println("This is Professor");
    }
}
class Lecturer extends Professor
{
    void message()
    {
        System.out.println("This is Lecturer");
    }
}
class Dynamic
{
    public static void main(String args[])
    {
        Principal x = new Principal();
        Professor y = new Professor();
        Lecturer z = new Lecturer();
        Principal ref; //reference variable of super class

        ref = x;        //called grand parent method
        ref.message();

        ref = y;        //called parent method
        ref.message();

        ref = z;        //called child method
        ref.message();
    }
}
```

"FINAL" KEYWORD

- **Use to declare any variable as *constant***

```
final int i=5;                    // value of i & pi can not be changed
final float pi=3.14;
```

- **Use to *prevent method overriding***

```
final void show()          // show() method can not override
```

- **Use to *prevent inheritance*** // Class A can not inherit

```
final class Parent
{
   ---
}
```

"STATIC" KEYWORD

- **Use to access members and methods of class by its class name <u>without creating object</u>.**

Eg: best example ' public static void main() ' method

- When objects of its class are declared, no copy of a static variable/method is created.
- **Methods declared as static have <u>several restrictions</u>:**

1) They can only call other static methods or variables.

2) They must only access static data.

3) They cannot refer to this or super in any way

- **E.g. 1) public static void main ()**

  2) static int a=10;

  3) static void show()

STATIC KEYWORD

```
//Demonstration of static members
  class MyWork
  {
     static int x = 10;
     static int count = 1;
     static void display()
     {
        System.out.println("Static has initialized...");
     }
     static void increment()
     {
        System.out.println("Function call : "+count);
        count++;
     }
  }
  class StaticMember
  {
     public static void main(String args[])
     {
        MyWork.display();                    //statement1
        System.out.print("Value of x: ");
        System.out.println(MyWork.x);        //statement2
        MyWork.increment();                  //statement3
        MyWork.increment();                  //statement4
        MyWork.increment();                  //statement5
     }
  }
```

"ABSTRACT" KEYWORD

- A class can be declared as abstract then its object can not be created .
- A method can be declared abstract then no implementation part present In its own class but only declaration part present
- If class contain one or more abstract method then class must be declared as abstract
- A class which extends abstract class must provide implementation of all abstract method of that class
- Syntax

```
abstract data_type method_name(arguments);
```

- Syntax

```
abstract class Abc
{
    abstract void display();
    . . . . . .
}
```

ABSTRACT KEYWORD

```
//Abstract class and method
abstract class Maths
{
    int var1, var2, var3;
    abstract void calculate();
    void addition()
    {
        var3 = var1 + var2;
        System.out.println("Addition : "+var3);
    }
}
class Arithmetic extends Maths
{
    Arithmetic(int x,int y)
    {
        var1 = x;
        var2 = y;
    }
    void calculate()
    {
        var3 = var1 - var2;
        System.out.println("Subtraction : "+var3);
    }
}
class AbstractClass
{
    public static void main(String args[])
    {
        Arithmetic a = new Arithmetic(30,18);
        a.calculate();
        a.addition();
    }
}
```

Output:

```
Subtraction : 12
Addition : 48
```